

GUION HABLADO · PARA LEER EN VIVO

Tower Defense Estocástico

Luciano · Slides 01 · 02 · 03 · 04 · 05 · 22

Lo que está en **azul** es lo que decís en voz alta. Lo **verde** es el enganche a la próxima slide. Lo **amarillo** es por si te preguntan. No tenés que decir todo: con lo azul alcanza.

FRASE ANCLA (repetila si te trabás): **“El entregable es la validación del modelo, el juego es el vehículo.”**

⚠ **Thaiel no viene:** hoy cubrís también su Wave 02 (slides 06→10). O sea: presentás **01→10 de corrido** y volvés para el cierre (22) después de la demo. Las slides de Thaiel están marcadas con borde naranja más abajo.

“Buenas, somos Luciano, Thael, Martín y Luca. Les traemos el **Tower Defense Estocástico**. Lo primero que queremos dejar en claro, porque si no se presta a confusión: **lo que entregamos no es un videojuego, es la validación de un modelo de colas**. El juego es la forma divertida de **ver** el modelo funcionando, pero el objeto de estudio es el modelo.

Tres palabras del título: es un sistema **estocástico**, o sea gobernado por el azar — los enemigos no llegan en momentos fijos ni las torres tardan siempre lo mismo, son variables aleatorias. Lo **modelamos** como una red de líneas de espera, y lo **simulamos** para estimar cómo se comporta. De ahí el nombre.

Aviso desde ya: al final hay **demo en vivo**, un Tower Defense real que reproduce la simulación. Y la charla es descontracturada — si surge algo, nos metemos.”

→ **Enganche:** “Antes de entrar en tema, les doy el mapa de por dónde vamos.”

Si preguntan “¿qué es un modelo?”: una representación simplificada del sistema real que conserva solo lo relevante para la pregunta que querés responder. Acá el sistema real es “defender una base de oleadas”; el modelo es una red de colas.

“La charla está dividida en **cuatro oleadas**, como el juego:

- **Wave 1** — el problema y el modelo de colas. Esto lo arranco yo.
- **Wave 2** — de dónde sale el azar y el motor de simulación.
- **Wave 3** — la validación: comparamos lo analítico contra lo simulado.
- **Wave 4** — experimentos, la arquitectura y la demo.

Cada oleada la lleva principalmente uno de nosotros, pero presentamos como equipo. Y este orden no es caprichoso: es el método clásico de un estudio de simulación — primero formulás el problema, después construís el modelo y sus entradas aleatorias, después lo **verificás y validás**, y al final experimentás y concluís.

Una distinción que vale la pena: **verificar** es “¿programé bien el modelo?”; **validar** es “¿el modelo representa la realidad?”. Las Waves 3 y 4 son exactamente eso.”

→ **Enganche:** “Antes del detalle, les muestro de un vistazo cómo está armada la máquina que van a ver corriendo.”

“Lo que van a ver corriendo en vivo se arma en **tres pasos**, y uno solo ocurre en pantalla.

Paso 1, offline: Python con la librería SimPy simula toda la batalla por **eventos discretos** y arma una línea de tiempo completa. Esto pasa **una sola vez**, antes de la presentación.

Paso 2, el contrato: esa simulación exporta un archivo, `output.json`, que es una **foto del sistema en cada instante** — cuántos hay en la cola, cuántos se fugaron, el estado de cada torre. Y como usamos una **semilla fija** para el azar, la corrida es **reproducible**: misma semilla, misma batalla, mismo archivo.

Paso 3, en vivo: Godot, el motor del juego, **lee ese JSON e interpola** para animar. El HUD le dibuja la teoría encima. La pantalla **no calcula nada** — cero operaciones de matemática durante la corrida.

¿Por qué importa? Porque separamos el motor pesado, que resuelve la dinámica del azar, del visor liviano, que solo reproduce. Así corre fluido hasta en una máquina humilde. Y en la demo van a ver que cambiamos de **escenario en caliente** — λ tranquilo, normal o saturado; más o menos torres; orden de llegada o prioridad — y el juego **recarga el archivo al instante**.”

→ **Enganche:** “Bueno, ahora sí, entremos en el problema concreto.”

Si preguntan “¿qué es simulación por eventos discretos?”: el reloj no avanza segundo a segundo; **salta de evento a evento** — un arribo, un fin de servicio, un sobrecalentamiento de torre. De esos saltos sale la línea de tiempo que guardamos.

Si preguntan “¿por qué guardar y no calcular en vivo?”: para que la demo sea **auditable y repetible** (misma semilla → mismo resultado) y para separar un cálculo pesado de una visualización liviana.

(Apoyate en el diagrama: *spawn* → *cola* → *c torres* → *base*.)

“El sistema es este: los enemigos aparecen y avanzan por un camino hacia nuestra **base**. En el medio hay una zona con **c torres** que los atacan. Si las torres están todas ocupadas, los enemigos **esperan en cola**. Y acá está el punto clave: si la zona está **llena** —no hay lugar ni en las torres ni en la cola— el enemigo **se fuga** y daña la base. A esa fuga después la vamos a llamar **bloqueo**.

Esto, que parece un juego, es un **sistema de líneas de espera** de manual: clientes que llegan, esperan si el servicio está ocupado, son atendidos y se van. Tiene sus tres piezas: cómo **llegan**, cómo se los **atiende**, y la **disciplina de la cola**.

Una cosa importante: nuestro sistema tiene **capacidad finita** —le decimos K —. Admite a lo sumo K enemigos entre la cola y las torres; el que llega y lo encuentra lleno, se pierde. Eso es lo que hace que existan fugas: es un modelo **con pérdida**. Y de vuelta el encuadre: el entregable es **validar este modelo**, el juego es el vehículo.”

→ **Enganche:** “Veamos cómo se traduce esto, pieza por pieza, al lenguaje de la teoría de colas.”

Si preguntan “¿población infinita?”: sí, asumimos población **infinita**: la tasa a la que llegan enemigos **no depende** de cuántos ya hay dentro del sistema.

(La más técnica. Andá tranquilo, leé la notación en voz alta y señalá la tabla.)

“Traduzcamos el juego a la teoría. Cada cosa del Tower Defense es algo de un sistema de colas:

- el **enemigo** es un **cliente**,
- la **torre** es un **servidor** o canal,
- **disparar hasta matarlo** es el **tiempo de servicio**,
- los **enemigos esperando** son la **cola**, que atendemos por orden de llegada,
- y la **fuga a la base** es el **bloqueo**: la cola está llena.

Todo esto se resume en una notación estándar para describir colas —se la conoce como **notación de Kendall-Lee**—. La nuestra es **(M / M / c) (FIFO / K / ∞)**. Se lee así:

- La primera **M** es cómo llegan: markoviano, llegadas tipo Poisson.
- La segunda **M** es el servicio: también markoviano, tiempos exponenciales, **sin memoria**.
- **c** es el número de torres, o sea de servidores.
- **FIFO** es la disciplina: el primero que llega es el primero en ser atendido.
- **K** es la capacidad total del sistema —cola más servicio— y es lo que habilita la fuga.
- El **∞** es la población: infinita.

Y el número que sostiene toda la Wave 3 es este: **$\rho = \lambda$ sobre c por μ** . Es el **factor de utilización**: la fracción de tiempo que, en promedio, cada torre está ocupada. La regla es **ρ menor que uno** para que el sistema sea estable; si ρ llega a uno o más, con cola infinita la fila crece sin límite — el sistema “no da abasto”.

Para que tengan los números: todo el TP sale de una corrida con $\lambda = 0,40$ arribos por segundo, $\mu = 0,25$ servicios por segundo y $c = 3$ torres. Eso da un ρ de **0,533** — y ojo, ese 0,533 es **cuenta directa** de λ sobre c por μ , no es simulación.”

→ **Enganche (ahora seguís vos con la Wave 02):** “Con el modelo planteado, sigamos: de dónde sale toda esta aleatoriedad y cómo la genera el motor de simulación.” (Si querés blanquearlo: “*Thaiel hoy no pudo venir, así que sigo yo.*”)

Símbolos: λ = tasa de llegadas (enemigos/s) · μ = tasa de servicio por torre (kills/s) · c = n° de torres · K = capacidad (cola+servicio) · $\rho = \lambda/(c \cdot \mu)$ = utilización.

Si preguntan “¿por qué Poisson y exponencial van juntos?”: si las llegadas son un proceso de Poisson de tasa λ , los **tiempos entre llegadas** son exponenciales. Lo mismo con el servicio y μ . Son dos caras de lo mismo.

COBERTURA · WAVE 02 (de Thael)

Las siguientes 5 slides (06→10) las presentás vos. Es la parte más técnica: **aleatoriedad, transformada inversa, SimPy y temperatura**. Apoyate en la **chuleta técnica** — acá está lo que decís en voz alta, ahí está el detalle de cada término. Idea fuerza de toda la Wave: **la temperatura rompe el supuesto markoviano** → **por eso hay que simular y no alcanza la fórmula**.

SLIDE 06 Variables del modelo

Cubre: Luciano (era Thael)

“Entremos en de qué está hecho el modelo. Tenemos **tres tipos de variables**:

- Las **aleatorias**: el tiempo entre llegadas, que es exponencial de tasa λ ; el tiempo de servicio de cada torre, exponencial de tasa μ ; y el tipo de enemigo, que es discreta.
- La **continua**: la **temperatura** de las torres. Es la gran variable continua del trabajo y a la vez el elemento de juego — cada torre se **calienta** al disparar y se **enfría** al descansar.
- Y las de **control y salida**: nosotros fijamos λ , μ , c , K y la semilla; y el modelo nos devuelve la cola media, la espera media, ρ y la tasa de fuga.

Y un chequeo que usamos para saber que los números son consistentes: la **Ley de Little** —una relación de la teoría de colas que dice que la cantidad media en el sistema es la tasa de llegada por el tiempo medio de permanencia, $L = \lambda$ por W —. Si nuestros números la cumplen, hay consistencia interna.”

→ **Enganche**: “Veamos de dónde sale esa aleatoriedad, o sea cómo generamos los números al azar.”

Clave técnica: discreta = valores contables (tipo de enemigo); **continua** = cualquier valor en un intervalo (temperatura, tiempos). La temperatura **no es aleatoria**: es determinística dado el historial de disparos.

“¿De dónde sale el azar? De un **generador de números pseudoaleatorios**. Se llama **pseudo** porque es 100% determinístico —una fórmula— pero la secuencia **parece** aleatoria. El del apunte es el **generador congruencial**: cada número sale del anterior con la fórmula $X(i+1) = (a \cdot X(i) + C) \bmod m$.

Lo que buscamos de un buen generador es que sea **uniforme**, **independiente**, **de período amplio** y, clave para nosotros, **reproducible**: misma semilla, misma secuencia.

Ahora, esos números salen entre 0 y 1, uniformes. Para convertirlos en **tiempos exponenciales** usamos el **método de la transformada inversa**: sorteás un número R uniforme, y despejás el tiempo de la fórmula. Para la exponencial queda $x = -(1/\lambda) \cdot \ln(R)$. Sorteás R , lees x . Y como la semilla está fija, sale siempre el mismo `output.json`.”

→ **Enganche**: “Con el azar resuelto, veamos el motor que arma toda la batalla: SimPy.”

Si preguntan por el generador real: en el apunte vemos el **congruencial** (está implementado en `prng.py`). La corrida final usa el generador por defecto del lenguaje, de período muy largo —es solo un **detalle de implementación**, no tema de clase—. Usamos **streams independientes** (uno para llegadas, otro para servicio, otro para tipo) para que no se correlacionen.

Si preguntan “¿qué es la transformada inversa?”: un método para generar una variable aleatoria con la distribución que querés, partiendo de un R uniforme entre 0 y 1: invertís su función de distribución acumulada.

(Esta es LA slide de la Wave. Si entendés esta, entendés por qué existe el TP.)

“El motor está hecho con SimPy, una librería de **simulación por eventos discretos**. La cola es FIFO; cada torre es un proceso que va sacando enemigos de esa cola y los atiende. La capacidad es finita — K —, así que el que llega con el sistema lleno se **fuga**: eso es el bloqueo.

Y acá está el corazón de todo: le agregamos **temperatura** a las torres. Cuando una torre se sobrecalienta, entra en **cooldown** y deja de atender un rato — o sea, baja la cantidad de torres efectivamente disponibles. Eso **rompe el supuesto markoviano**, que asume que las c torres están siempre disponibles. Por eso la fórmula de colas nos da solo una **cota** optimista, y para tener el número real **hay que simular**. Esa es la razón de ser del trabajo.”

→ **Enganche:** “Y todo esto, ¿cómo se ve en el código? Lo miramos en dos pantallas.”
(Cue de demo: las torres pasan de azul a rojo y se apagan en cooldown.)

Si preguntan “¿qué es el supuesto markoviano?”: que el sistema no tiene memoria — llegadas Poisson y servicio exponencial, con las c torres siempre activas. La temperatura le mete “memoria” (depende del historial de disparos), así que deja de ser un M/M/c puro.

(No leas el código línea por línea. Mostrá que cada línea ES un concepto.)

“Lo importante de este código es que **cada línea es un concepto de la materia**, no código suelto.

Arriba está la función de **llegadas**: sortea con la exponencial cuándo llega el próximo enemigo —eso **es** un proceso de Poisson—; si el sistema ya está lleno, en capacidad K , lo cuenta como **fuga**, que es el bloqueo; y si hay lugar, lo mete en la cola FIFO.

Abajo está el **proceso de cada torre**: toma el siguiente de la cola —y si está vacía, espera—, y lo atiende durante un tiempo exponencial de tasa μ , que es el disparo hasta matarlo. En dos bloques chiquitos, el programa **implementa exactamente** el modelo $(M/M/c)(FIFO/K/\infty)$ que planteamos al principio.”

→ **Enganche:** “Y la segunda mitad: la temperatura y el motor que pone todo a correr.”

“Acá está la **temperatura**, que es la variable continua de estado. Tiene dos tramos: una **rampa** mientras la torre dispara, y cuando descansa **decae** hacia la temperatura ambiente —ese enfriamiento gradual es lo que se conoce como la ley de enfriamiento de Newton: cuanto más caliente está, más rápido baja—. Cuando supera el máximo, la torre entra en **cooldown** y deja de atender: ahí baja la capacidad efectiva y se rompe el supuesto markoviano.

Y abajo está el **motor**, la función que pone todo a andar: lanza el proceso de llegadas, lanza un proceso por cada torre, y corre hasta el tiempo final. El reloj no avanza segundo a segundo: **salta de evento a evento**, procesando la lista de eventos futuros. Y volvemos a la idea de toda la Wave: **por esto hay que simular** — la fórmula sola no ve la temperatura.”

→ **Enganche (a Martín):** “Con el motor armado, le paso a Martín, que va a mostrar que esto efectivamente está bien — la validación.”

Si preguntan por “híbrida”: la simulación combina **eventos discretos** (llegadas, fines de servicio) con una **variable continua** (temperatura) que evoluciona entre eventos. Por eso se la puede llamar simulación híbrida.

(Volvés vos al final, después de la demo. Tono tranquilo de cierre.)

“Y con eso cerramos. La frase con la que arrancamos vale para terminar: **defendemos el modelo, no el juego — pero el juego lo hace divertido.**

En una sola charla recorrimos el ciclo completo: **modelar** un sistema estocástico, **generarlo** con números pseudoaleatorios, **validarlo** contra la teoría de colas, y usar la simulación para **responder una pregunta de diseño**: ¿cuántas torres conviene poner?

Acá están los cuatro repositorios —la documentación, el backend en Python, el frontend en Godot y esta presentación—. Y quedamos para las preguntas. Gracias.”

Pregunta casi segura — “¿por qué simular y no usar la fórmula?”: porque le agregamos **temperatura** a las torres (se calientan y se enfrían), y eso rompe el supuesto markoviano: el servicio deja de ser “sin memoria”. Cuando no hay fórmula cerrada exacta, simulás. Igual validamos contra la fórmula en el caso sin temperatura, para mostrar que el motor está bien.

Pregunta casi segura — “¿cómo sabés que está bien?”: por dos lados. Primero, una relación de la teoría de colas que vincula la cantidad media en el sistema con la tasa de llegada y el tiempo medio de permanencia —se la conoce como **Ley de Little**—; nuestros números la cumplen. Segundo, sin temperatura la simulación **coincide con el modelo M/M/c teórico**. Eso es la validación.

Tips rápidos para hoy

- **Tu ancla es una sola frase**, repetila si te trabás: “**el entregable es la validación del modelo, el juego es el vehículo.**” Volvé a eso y reorientás.
- **En la 05**, si te ponés nervioso con la notación, leela de izquierda a derecha como una lista. No la memorices: está en pantalla, andá señalando.
- **No tenés que decir todo.** Lo azul es lo mínimo; lo amarillo es solo por si preguntan. Si vas corto de tiempo, las negritas alcanzan.
- **Respiración:** cada slide arrancá con una frase corta antes de la explicación larga.
- **Números de memoria:** $\lambda=0,40 \cdot \mu=0,25 \cdot c=3 \cdot K=10 \cdot p=0,533 \cdot$ semilla 42 · 1200 s.